

App Functionalities, Workflows, APIs, Tech Stack & Implementation Blueprint

Project

Lebanon Local Commerce & Delivery Platform

This file explains the main app functionalities, transitions, workflows, APIs, recommended tech stack, integrations, and implementation notes needed before starting design and development.

1. Purpose of This Document

The goal of this document is to make the app clear before development starts.

It defines:

- What functionalities will exist in the app.
- How the four systems connect together.
- How users move from one screen to another.
- What workflows happen in the background.
- What APIs are needed.
- What tech stack is recommended.
- What third-party services may be required.
- What should be included in the MVP.
- What should be delayed for future versions.

The platform has four main systems:

1. **Client System**
2. **Store System**
3. **Service Provider System**
4. **Delivery System**

It also requires one important control system:

1. **Admin System**

All systems are connected through one shared backend, one database, and role-based authorization.

2. Big Picture System Flow

```
Client
↓
Places order / requests service / requests custom delivery
↓
Store or Service Provider receives request
↓
Store/Provider accepts, rejects, prepares, or quotes
↓
Delivery is created if needed
↓
Driver accepts or is assigned
↓
Driver picks up and delivers
↓
Payment is confirmed
↓
Receipt is generated
↓
Admin can monitor everything
```

The main business logic is built around:

- Orders
- Service requests
- Deliveries
- Inventory
- Payments
- Receipts
- Notifications
- Authorization

3. Applications Needed

3.1 Client Mobile App

Used by customers to order products, request services, request delivery, track orders, and view receipts.

3.2 Driver Mobile App

Used by delivery drivers to accept jobs, update delivery status, navigate, and manage earnings.

3.3 Store / Service Provider Dashboard

Used by businesses to manage products, services, orders, requests, inventory, receipts, and delivery options.

This can start as a responsive web dashboard.

3.4 Admin Dashboard

Used by platform admins to control users, stores, providers, drivers, orders, deliveries, fees, receipts, support, and reports.

4. User Roles

Main Roles

Admin
Support Agent
Client
Store Owner
Store Staff
Store Driver
Service Provider Owner
Service Provider Staff
Independent Driver

Important Rule

One user can have multiple roles.

Example:

Mohamad can be:

- Client
- Store Owner
- Independent Driver

After login, if the user has multiple roles, they should see a role selection screen:

Continue as Client
Continue as Store Owner
Continue as Driver

5. Core Functionalities by System

5.1 Client System Functionalities

Account & Profile

- Register with phone/email.
- Login.
- Logout.
- Forgot password.
- Edit profile.
- Add profile photo.
- Manage addresses.
- Save multiple addresses.
- Choose default address.

Home & Discovery

- View nearby stores.
- View nearby restaurants.
- View service providers.
- Search for products.
- Search for stores.
- Search for services.
- View categories.
- View offers/promotions later.
- View recently ordered stores.

Store Ordering

- Open store page.
- View store details.
- View product categories.
- View products.
- View product details.
- Add item to cart.
- Change item quantity.

- Remove item from cart.
- Add order notes.
- Choose delivery address.
- Choose payment method.
- Place order.
- Track order status.
- Cancel order if still pending.
- Contact store.
- Contact driver after driver assigned.
- View receipt.
- Rate order/store/driver.

Service Requests

- Browse service providers.
- Open provider page.
- View services.
- Choose service.
- Fill request form.
- Add description.
- Upload image if needed later.
- Choose location.
- Request quote.
- Accept/reject quote.
- Track service request.
- Contact provider.
- Request pickup/delivery if needed.
- View receipt.
- Rate provider.

Custom Delivery

- Create pickup and dropoff request.
- Add pickup location.
- Add dropoff location.
- Add item description.
- Add notes.
- See estimated delivery fee.
- Confirm request.
- Track driver.
- Contact driver.
- Confirm delivery.
- View receipt.

Notifications

- Order accepted.

- Order rejected.
- Order preparing.
- Driver assigned.
- Driver picked up order.
- Driver on the way.
- Order delivered.
- Quote received.
- Payment confirmed.
- Issue update.

Receipts & History

- View order history.
- View service request history.
- View custom delivery history.
- View receipt details.
- Download/share receipt later.

Support

- Report order issue.
- Report delivery issue.
- Report payment issue.
- Contact support.
- View support ticket status.

5.2 Store System Functionalities

Store Account & Profile

- Store owner registration.
- Store login.
- Store profile setup.
- Business name.
- Business category.
- Phone number.
- Address.
- Map location.
- Opening hours.
- Store logo/image.
- Delivery options.
- Store approval status.

Product Management

- Add product.
- Edit product.
- Delete product.
- Upload product image.
- Assign category.
- Set product price.
- Set product description.
- Mark product available/unavailable.
- Add product variants later.

Inventory Management

- Set product quantity.
- Add stock.
- Remove stock.
- Manual adjustment.
- Set low-stock alert.
- View low-stock products.
- View out-of-stock products.
- Auto-reduce stock when order is accepted/completed.
- Restore stock if order is cancelled.
- View inventory movement history.

Order Management

- View new orders.
- Accept order.
- Reject order with reason.
- Mark order as preparing.
- Mark order ready for pickup.
- Assign store driver.
- Request independent driver.
- Contact client.
- Contact driver.
- View order details.
- View order timeline.
- View payment status.
- Complete order.

Receipts

- View receipts.
- Filter by day/week/month.
- View receipt details.
- See cash collected by driver.

- See order total and delivery fee.
- Download/export later.

Delivery Settings

- Store delivery only.
- Independent platform delivery only.
- Both store and platform delivery.
- Pickup only.
- Set delivery preparation time.
- Manage store drivers.

Staff Management

- Add staff user.
- Assign permissions.
- Remove staff.
- Staff role: order manager.
- Staff role: inventory manager.
- Staff role: cashier later.

Reports

- Daily orders.
 - Daily sales.
 - Best-selling products.
 - Low-stock products.
 - Cancelled orders.
 - Delivery performance.
-

5.3 Service Provider System Functionalities

Provider Account & Profile

- Provider registration.
- Provider login.
- Provider profile setup.
- Business/service name.
- Service category.
- Phone number.
- Address.
- Map location.
- Opening hours.
- Approval status.

Service Management

- Add service.
- Edit service.
- Delete service.
- Set service description.
- Set pricing type.
- Set base price.
- Mark service available/unavailable.

Service Request Management

- View new service requests.
- Accept request.
- Reject request.
- Send quote.
- Update quote.
- Mark quote accepted.
- Mark service in progress.
- Mark service ready.
- Mark service completed.
- Contact client.
- Request delivery if needed.

Delivery for Services

- Request pickup from client.
- Request return delivery to client.
- Track delivery.
- Contact driver.
- Confirm item received.
- Confirm item returned.

Receipts

- Generate service receipt.
- View service receipts.
- View payment status.
- Filter receipts.

Reports

- Daily service requests.
- Completed jobs.
- Cancelled jobs.
- Total earnings.
- Most requested services.

5.4 Delivery System Functionalities

Driver Account & Profile

- Driver registration.
- Driver login.
- Driver profile setup.
- Upload documents later.
- Vehicle type.
- Phone number.
- Availability status.
- Approval status.

Job Discovery

- View available delivery jobs.
- View pickup location.
- View dropoff location.
- View estimated distance.
- View estimated fee.
- Accept job.
- Reject job.

Active Delivery

- Navigate to pickup.
- Mark arrived at pickup.
- Mark picked up.
- Navigate to dropoff.
- Mark arrived at destination.
- Mark delivered.
- Mark failed delivery.
- Contact client.
- Contact store/provider.
- Contact support.

Cash Collection

- See amount to collect.
- Mark cash collected.
- Confirm payment.
- View collected cash history.

Earnings

- View delivery earnings.
- View completed deliveries.
- View cancelled/failed deliveries.
- View daily/weekly summary.

Driver Notifications

- New delivery job.
 - Job cancelled.
 - Pickup changed.
 - Payment collection required.
 - Support message.
-

5.5 Admin System Functionalities

Admin Overview

- Total users.
- Total stores.
- Total service providers.
- Total drivers.
- Active orders.
- Active deliveries.
- Daily revenue.
- Open support tickets.

User Management

- View users.
- Search users.
- View user details.
- Suspend user.
- Activate user.
- View user roles.

Store Management

- View stores.
- Approve store.
- Reject store.
- Suspend store.
- Edit store category.

- View store orders.
- View store receipts.

Service Provider Management

- View providers.
- Approve provider.
- Reject provider.
- Suspend provider.
- View provider requests.
- View provider receipts.

Driver Management

- View drivers.
- Approve driver.
- Reject driver.
- Suspend driver.
- View active drivers.
- View driver delivery history.
- View driver cash collection.

Order & Delivery Monitoring

- View all orders.
- View all service requests.
- View all deliveries.
- Filter by status.
- Handle failed deliveries.
- Reassign delivery.
- Cancel order if needed.

Finance & Receipts

- View payments.
- View receipts.
- View cash collection.
- Track delivery fees.
- Track commissions later.
- Export reports later.

Platform Settings

- Manage zones.
- Manage categories.
- Manage delivery fees.
- Manage commission rates.

- Manage support reasons.
- Manage cancellation reasons.

Support

- View support tickets.
 - Assign ticket to support agent.
 - Reply to ticket.
 - Close ticket.
 - Escalate issue.
-

6. App Transitions and Navigation

This section explains how users move through the app.

6.1 Client App Transitions

First-Time User Flow

```
Splash Screen
↓
Onboarding Screens
↓
Register / Login
↓
Location Permission
↓
Add Address
↓
Home Screen
```

Returning User Flow

```
Splash Screen
↓
Check authentication token
↓
If logged in: Home Screen
If not logged in: Login Screen
```

Multi-Role User Flow

```
Splash Screen
↓
Login
↓
Role Selection
↓
Selected Dashboard
```

Example:

```
Login
↓
Choose: Client / Store Owner / Driver
↓
Open selected system
```

Store Order Flow Screens

```
Home
↓
Store Listing
↓
Store Details
↓
Product Details
↓
Cart
↓
Checkout
↓
Order Created
↓
Order Tracking
↓
Receipt
↓
Rating
```

Service Request Flow Screens

```
graph TD;
    Home --> ServiceProviderListing[Service Provider Listing];
    ServiceProviderListing --> ProviderDetails[Provider Details];
    ProviderDetails --> ServiceDetails[Service Details];
    ServiceDetails --> ServiceRequestForm[Service Request Form];
    ServiceRequestForm --> RequestTracking[Request Tracking];
    RequestTracking --> QuoteScreen[Quote Screen, if needed];
    QuoteScreen --> ServiceProgress[Service Progress];
    ServiceProgress --> Receipt[Receipt];
    Receipt --> Rating[Rating];
```

Home
↓
Service Provider Listing
↓
Provider Details
↓
Service Details
↓
Service Request Form
↓
Request Tracking
↓
Quote Screen, if needed
↓
Service Progress
↓
Receipt
↓
Rating

Custom Delivery Flow Screens

```
graph TD;
    Home --> CustomDeliveryRequest[Custom Delivery Request];
    CustomDeliveryRequest --> PickupLocation[Pickup Location];
    PickupLocation --> DropoffLocation[Dropoff Location];
    DropoffLocation --> ItemDetails[Item Details];
    ItemDetails --> DeliveryFeeEstimate[Delivery Fee Estimate];
    DeliveryFeeEstimate --> ConfirmRequest[Confirm Request];
    ConfirmRequest --> DriverMatching[Driver Matching];
    DriverMatching --> LiveTracking[Live Tracking];
    LiveTracking --> Receipt[Receipt];
```

Home
↓
Custom Delivery Request
↓
Pickup Location
↓
Dropoff Location
↓
Item Details
↓
Delivery Fee Estimate
↓
Confirm Request
↓
Driver Matching
↓
Live Tracking
↓
Receipt

6.2 Store Dashboard Transitions

Store Login Flow

```
Login
  ↓
If store approved: Store Dashboard
If not approved: Pending Approval Screen
```

New Order Flow

```
Dashboard
  ↓
New Orders
  ↓
Order Details
  ↓
Accept / Reject
  ↓
Preparing
  ↓
Ready for Pickup
  ↓
Assign Driver / Request Driver
  ↓
Delivery Tracking
  ↓
Receipt
```

Product & Inventory Flow

```
Dashboard
  ↓
Products
  ↓
Add/Edit Product
  ↓
Inventory
  ↓
Stock Adjustment
```

↓
Inventory Movement History

6.3 Service Provider Transitions

Service Request Flow

Provider Dashboard
↓
New Requests
↓
Request Details
↓
Accept / Reject
↓
Send Quote, if needed
↓
Client Accepts Quote
↓
In Progress
↓
Ready / Completed
↓
Receipt

Delivery Needed Flow

Request Details
↓
Request Delivery
↓
Pickup/Dropoff Details
↓
Driver Assigned
↓
Delivery Tracking
↓
Item Received / Returned

6.4 Driver App Transitions

Driver Login Flow

```
Login
↓
If approved: Driver Home
If not approved: Pending Approval Screen
```

Delivery Job Flow

```
Driver Home
↓
Set Available
↓
Available Jobs
↓
Job Details
↓
Accept Job
↓
Navigate to Pickup
↓
Arrived at Pickup
↓
Picked Up
↓
Navigate to Dropoff
↓
Arrived at Destination
↓
Delivered
↓
Cash Collection, if required
↓
Delivery Completed
↓
Earnings Updated
```

6.5 Admin Dashboard Transitions

Admin Monitoring Flow

```
Admin Login
  ↓
Admin Overview
  ↓
Orders / Deliveries / Users / Stores / Providers
  ↓
Details Page
  ↓
Take Action
  ↓
Audit Log Updated
```

7. Workflow State Machines

State machines are important because every order, service request, and delivery must move through controlled statuses.

7.1 Product Order State Machine

```
pending
  ↓
accepted_by_store
  ↓
preparing
  ↓
ready_for_pickup
  ↓
driver_assigned
  ↓
picked_up
  ↓
on_the_way
  ↓
delivered
```

↓
completed

Alternative paths:

```
pending → rejected_by_store  
pending → cancelled_by_client  
accepted_by_store → cancelled_by_store/admin  
picked_up → failed_delivery  
any_active_status → issue_reported
```

7.2 Service Request State Machine

```
pending  
↓  
accepted  
↓  
waiting_for_quote, optional  
↓  
quote_sent  
↓  
quote_accepted  
↓  
in_progress  
↓  
ready  
↓  
completed
```

Alternative paths:

```
pending → rejected  
quote_sent → quote_rejected  
any_active_status → cancelled  
any_active_status → issue_reported
```

7.3 Delivery State Machine

```
waiting_for_driver
  ↓
driver_assigned
  ↓
arrived_at_pickup
  ↓
picked_up
  ↓
on_the_way
  ↓
arrived_at_destination
  ↓
delivered
```

Alternative paths:

```
waiting_for_driver → cancelled
picked_up → failed_delivery
any_active_status → issue_reported
```

7.4 Payment State Machine

```
unpaid
  ↓
pending_collection
  ↓
collected
  ↓
paid
```

Alternative paths:

```
unpaid → cancelled
collected → disputed
paid → refunded, later
```

8. API Structure

All systems should use one backend API.

Recommended API base format:

```
/api/v1
```

Example:

```
/api/v1/auth/login  
/api/v1/stores  
/api/v1/orders  
/api/v1/deliveries
```

9. Authentication APIs

Register

```
POST /api/v1/auth/register
```

Purpose:

Create a new user account.

Body example:

```
{  
  "full_name": "Mohamad Serhal",  
  "phone": "+96100000000",  
  "email": "mohamad@example.com",  
  "password": "password"  
}
```

Login

```
POST /api/v1/auth/login
```

Purpose:

Authenticate user and return access token.

Body example:

```
{  
  "phone": "+96100000000",  
  "password": "password"  
}
```

Get Current User

```
GET /api/v1/auth/me
```

Purpose:

Return logged-in user, roles, and permissions.

Refresh Token

```
POST /api/v1/auth/refresh
```

Logout

```
POST /api/v1/auth/logout
```

10. User & Role APIs

```
GET    /api/v1/users/me
PATCH /api/v1/users/me
GET    /api/v1/users/me/roles
POST   /api/v1/users/me/active-role
```

Admin-only:

```
GET    /api/v1/admin/users
GET    /api/v1/admin/users/:id
PATCH /api/v1/admin/users/:id/status
POST   /api/v1/admin/users/:id/roles
DELETE /api/v1/admin/users/:id/roles/:roleId
```

11. Client APIs

Address APIs

```
GET    /api/v1/client/addresses
POST   /api/v1/client/addresses
PATCH /api/v1/client/addresses/:id
DELETE /api/v1/client/addresses/:id
POST   /api/v1/client/addresses/:id/default
```

Client Home APIs

```
GET /api/v1/client/home
GET /api/v1/client/categories
GET /api/v1/client/search?q=keyword
```

12. Store APIs

Public Store APIs

Used by clients.

```
GET /api/v1/stores
GET /api/v1/stores/:id
GET /api/v1/stores/:id/products
GET /api/v1/stores/:id/products/:productId
```

Store Owner APIs

```
POST /api/v1/stores
GET /api/v1/store-owner/stores
GET /api/v1/store-owner/stores/:id
PATCH /api/v1/store-owner/stores/:id
PATCH /api/v1/store-owner/stores/:id/delivery-settings
GET /api/v1/store-owner/stores/:id/dashboard
```

Store Staff APIs

```
GET /api/v1/store/staff/orders
GET /api/v1/store/staff/orders/:id
PATCH /api/v1/store/staff/orders/:id/status
```

13. Product APIs

```
GET /api/v1/store-owner/stores/:storeId/products
POST /api/v1/store-owner/stores/:storeId/products
GET /api/v1/store-owner/stores/:storeId/products/:productId
PATCH /api/v1/store-owner/stores/:storeId/products/:productId
DELETE /api/v1/store-owner/stores/:storeId/products/:productId
PATCH /api/v1/store-owner/stores/:storeId/products/:productId/availability
```

14. Inventory APIs

```
GET /api/v1/store-owner/stores/:storeId/inventory
GET /api/v1/store-owner/stores/:storeId/inventory/low-stock
GET /api/v1/store-owner/stores/:storeId/inventory/out-of-stock
PATCH /api/v1/store-owner/stores/:storeId/inventory/:productId
```

```
POST /api/v1/store-owner/stores/:storeId/inventory/:productId/adjust
GET /api/v1/store-owner/stores/:storeId/inventory-movements
```

Adjust Inventory Body Example

```
{
  "quantity_change": 20,
  "reason": "New stock received"
}
```

For removing stock:

```
{
  "quantity_change": -5,
  "reason": "Damaged items"
}
```

15. Order APIs

Client Order APIs

```
POST /api/v1/client/orders
GET /api/v1/client/orders
GET /api/v1/client/orders/:id
POST /api/v1/client/orders/:id/cancel
GET /api/v1/client/orders/:id/receipt
```

Create Order Body Example

```
{
  "store_id": "store_123",
  "address_id": "address_456",
  "payment_method": "cash_on_delivery",
  "notes": "Call before arriving",
  "items": [
    {
      "product_id": "product_1",
      "quantity": 2
    },
  ],
}
```

```
{
  "product_id": "product_2",
  "quantity": 1
}
]
```

Store Order APIs

```
GET /api/v1/store-owner/stores/:storeId/orders
GET /api/v1/store-owner/stores/:storeId/orders/:orderId
POST /api/v1/store-owner/stores/:storeId/orders/:orderId/accept
POST /api/v1/store-owner/stores/:storeId/orders/:orderId/reject
POST /api/v1/store-owner/stores/:storeId/orders/:orderId/mark-preparing
POST /api/v1/store-owner/stores/:storeId/orders/:orderId/mark-ready
POST /api/v1/store-owner/stores/:storeId/orders/:orderId/request-delivery
POST /api/v1/store-owner/stores/:storeId/orders/:orderId/assign-store-driver
```

16. Service Provider APIs

Public Provider APIs

```
GET /api/v1/service-providers
GET /api/v1/service-providers/:id
GET /api/v1/service-providers/:id/services
GET /api/v1/service-providers/:id/services/:serviceId
```

Provider Owner APIs

```
POST /api/v1/provider-owner/providers
GET /api/v1/provider-owner/providers
GET /api/v1/provider-owner/providers/:id
PATCH /api/v1/provider-owner/providers/:id
GET /api/v1/provider-owner/providers/:id/dashboard
```

17. Service APIs

```
GET    /api/v1/provider-owner/providers/:providerId/services
POST   /api/v1/provider-owner/providers/:providerId/services
GET    /api/v1/provider-owner/providers/:providerId/services/:serviceId
PATCH /api/v1/provider-owner/providers/:providerId/services/:serviceId
DELETE /api/v1/provider-owner/providers/:providerId/services/:serviceId
PATCH /api/v1/provider-owner/providers/:providerId/services/:serviceId/
availability
```

18. Service Request APIs

Client Service Request APIs

```
POST /api/v1/client/service-requests
GET  /api/v1/client/service-requests
GET  /api/v1/client/service-requests/:id
POST /api/v1/client/service-requests/:id/cancel
POST /api/v1/client/service-requests/:id/accept-quote
POST /api/v1/client/service-requests/:id/reject-quote
GET  /api/v1/client/service-requests/:id/receipt
```

Create Service Request Body Example

```
{
  "provider_id": "provider_123",
  "service_id": "service_456",
  "address_id": "address_789",
  "description": "Need phone screen replacement",
  "preferred_time": "2026-05-02T10:00:00Z"
}
```

Provider Service Request APIs

```
GET    /api/v1/provider-owner/providers/:providerId/service-requests
GET    /api/v1/provider-owner/providers/:providerId/service-requests/:requestId
POST   /api/v1/provider-owner/providers/:providerId/service-requests/:requestId/
accept
POST   /api/v1/provider-owner/providers/:providerId/service-requests/:requestId/
```

```
reject
POST /api/v1/provider-owner/providers/:providerId/service-requests/:requestId/
send-quote
POST /api/v1/provider-owner/providers/:providerId/service-requests/:requestId/
mark-in-progress
POST /api/v1/provider-owner/providers/:providerId/service-requests/:requestId/
mark-ready
POST /api/v1/provider-owner/providers/:providerId/service-requests/:requestId/
complete
POST /api/v1/provider-owner/providers/:providerId/service-requests/:requestId/
request-delivery
```

19. Delivery APIs

Driver APIs

```
GET /api/v1/driver/profile
PATCH /api/v1/driver/profile
POST /api/v1/driver/availability
POST /api/v1/driver/location
GET /api/v1/driver/jobs/available
GET /api/v1/driver/jobs/:id
POST /api/v1/driver/jobs/:id/accept
POST /api/v1/driver/jobs/:id/reject
POST /api/v1/driver/deliveries/:id/arrived-pickup
POST /api/v1/driver/deliveries/:id/picked-up
POST /api/v1/driver/deliveries/:id/on-the-way
POST /api/v1/driver/deliveries/:id/arrived-destination
POST /api/v1/driver/deliveries/:id/delivered
POST /api/v1/driver/deliveries/:id/failed
POST /api/v1/driver/deliveries/:id/cash-collected
GET /api/v1/driver/deliveries/history
GET /api/v1/driver/earnings
```

Store/Provider Delivery APIs

```
POST /api/v1/deliveries/request
GET /api/v1/deliveries/:id
GET /api/v1/deliveries/:id/tracking
POST /api/v1/deliveries/:id/cancel
```

Custom Delivery APIs

```
POST /api/v1/client/custom-deliveries
GET /api/v1/client/custom-deliveries
GET /api/v1/client/custom-deliveries/:id
POST /api/v1/client/custom-deliveries/:id/cancel
```

20. Receipt APIs

```
GET /api/v1/client/receipts
GET /api/v1/client/receipts/:id
GET /api/v1/store-owner/stores/:storeId/receipts
GET /api/v1/provider-owner/providers/:providerId/receipts
GET /api/v1/admin/receipts
GET /api/v1/admin/receipts/:id
```

Future:

```
GET /api/v1/receipts/:id/download-pdf
POST /api/v1/receipts/:id/send-email
```

21. Payment APIs

MVP payment method:

Cash on delivery

APIs:

```
GET /api/v1/payments/:id
POST /api/v1/payments/:id/mark-collected
POST /api/v1/payments/:id/mark-paid
GET /api/v1/admin/payments
GET /api/v1/driver/payments/collected-cash
```

Future online payments:

```
POST /api/v1/payments/create-intent
POST /api/v1/payments/webhook
POST /api/v1/payments/refund
```

22. Notification APIs

```
GET /api/v1/notifications
POST /api/v1/notifications/:id/read
POST /api/v1/notifications/read-all
```

Push notifications should be sent automatically for:

- New order.
- Order status change.
- New delivery job.
- Driver assigned.
- Delivery status update.
- Quote sent.
- Payment collected.
- Support ticket update.

23. Messaging APIs

MVP can start with simple contact buttons and order notes.

Full in-app messaging can be added later.

Future APIs:

```
GET /api/v1/conversations
POST /api/v1/conversations
GET /api/v1/conversations/:id/messages
POST /api/v1/conversations/:id/messages
```

Conversation types:

```
client_store
client_driver
```

```
store_driver
client_provider
provider_driver
support
```

24. Rating APIs

```
POST /api/v1/client/ratings
GET /api/v1/stores/:id/ratings
GET /api/v1/service-providers/:id/ratings
GET /api/v1/drivers/:id/ratings
```

Rating targets:

```
store
service_provider
driver
order
service_request
delivery
```

25. Support APIs

```
POST /api/v1/support/tickets
GET /api/v1/support/tickets
GET /api/v1/support/tickets/:id
POST /api/v1/support/tickets/:id/reply
POST /api/v1/support/tickets/:id/close
```

Admin support:

```
GET /api/v1/admin/support/tickets
GET /api/v1/admin/support/tickets/:id
POST /api/v1/admin/support/tickets/:id/assign
POST /api/v1/admin/support/tickets/:id/reply
POST /api/v1/admin/support/tickets/:id/close
```

26. Admin APIs

```
GET /api/v1/admin/dashboard
GET /api/v1/admin/users
GET /api/v1/admin/stores
GET /api/v1/admin/providers
GET /api/v1/admin/drivers
GET /api/v1/admin/orders
GET /api/v1/admin/service-requests
GET /api/v1/admin/deliveries
GET /api/v1/admin/receipts
GET /api/v1/admin/payments
```

Approval actions:

```
POST /api/v1/admin/stores/:id/approve
POST /api/v1/admin/stores/:id/reject
POST /api/v1/admin/providers/:id/approve
POST /api/v1/admin/providers/:id/reject
POST /api/v1/admin/drivers/:id/approve
POST /api/v1/admin/drivers/:id/reject
```

Management actions:

```
PATCH /api/v1/admin/stores/:id/status
PATCH /api/v1/admin/providers/:id/status
PATCH /api/v1/admin/drivers/:id/status
PATCH /api/v1/admin/users/:id/status
```

Settings:

```
GET /api/v1/admin/categories
POST /api/v1/admin/categories
PATCH /api/v1/admin/categories/:id
DELETE /api/v1/admin/categories/:id

GET /api/v1/admin/zones
POST /api/v1/admin/zones
PATCH /api/v1/admin/zones/:id
DELETE /api/v1/admin/zones/:id
```

```
GET    /api/v1/admin/fees
PATCH /api/v1/admin/fees
```

27. Realtime Events

Realtime updates are important for orders and delivery.

Recommended options:

- WebSockets
- Socket.IO
- Firebase Realtime features later if needed

Client Realtime Events

```
order.status.updated
delivery.driver.assigned
delivery.location.updated
delivery.status.updated
service_request.status.updated
quote.sent
payment.status.updated
```

Store Realtime Events

```
order.created
order.cancelled
delivery.driver.assigned
delivery.status.updated
payment.collected
inventory.low_stock
```

Driver Realtime Events

```
delivery.job.created
delivery.job.cancelled
delivery.assigned
delivery.details.updated
```

Admin Realtime Events

```
order.created  
delivery.failed  
support.ticket.created  
payment.disputed
```

28. Recommended Tech Stack

28.1 Mobile Apps

Recommended:

```
Flutter
```

Why:

- One codebase for Android and iOS.
- Good performance.
- Strong for mobile UI.
- Good for maps and location.
- Good for apps that may start Android-first.

Alternative:

```
React Native
```

React Native is also good, especially if the team is already strong in React.

Recommendation

Use **Flutter** if you want a polished mobile-first app.

Use **React Native** if you want to share knowledge with React dashboards.

28.2 Web Dashboards

Recommended:

```
Next.js + React + TypeScript
```

Why:

- Strong for dashboards.
- Good routing.
- Good performance.
- Easy to build admin panels.
- Large ecosystem.

UI options:

```
Tailwind CSS  
Shadcn UI  
Material UI  
Ant Design
```

Recommended for clean dashboard:

```
Next.js + TypeScript + Tailwind CSS + Shadcn UI
```

28.3 Backend

Recommended:

```
NestJS + TypeScript
```

Why:

- Modular architecture.
- Good for large systems.
- Strong TypeScript support.
- Good with PostgreSQL.
- Good for role-based authorization.
- Good for REST APIs and WebSockets.

Alternative options:

Laravel
Django
FastAPI
Express.js

Recommended final choice:

NestJS

28.4 Database

Recommended:

PostgreSQL

Why:

- Reliable.
- Good for business data.
- Good for relationships.
- Good for orders, receipts, payments, inventory.
- Better than NoSQL for this type of platform.

ORM options:

Prisma
TypeORM
Drizzle

Recommended:

Prisma

Why:

- Clean schema.
- Easy migrations.

- Good TypeScript support.
-

28.5 Realtime & Cache

Recommended:

Redis
Socket.IO

Redis usage:

- Driver online status.
- Driver live location cache.
- Job matching queues.
- Rate limiting.
- Temporary OTP storage.

Socket.IO usage:

- Order updates.
 - Delivery tracking.
 - New job notifications.
 - Live status updates.
-

28.6 Push Notifications

Recommended:

Firebase Cloud Messaging

Used for:

- Client notifications.
 - Driver job alerts.
 - Store order alerts.
 - Provider request alerts.
-

28.7 Maps & Location

Options:

Google Maps
Mapbox
OpenStreetMap

Recommended for MVP:

Google Maps or Mapbox

Needed features:

- Address selection.
- Map pin.
- Driver navigation.
- Distance calculation.
- Delivery tracking.
- Zones later.

Important note:

Map APIs can become expensive at scale, so usage should be monitored.

28.8 File/Image Storage

Options:

AWS S3
Cloudflare R2
Firebase Storage
Supabase Storage

Used for:

- Product images.
- Store logos.
- Provider images.
- Driver documents later.
- Support attachments later.

Recommended:

Cloudflare R2 or AWS S3

28.9 Hosting

Backend hosting options:

Render
Railway
DigitalOcean
AWS
Hetzner

Database hosting options:

Supabase Postgres
Neon Postgres
Railway Postgres
AWS RDS
DigitalOcean Managed PostgreSQL

Recommended MVP setup:

Backend: Render / Railway / DigitalOcean
Database: Supabase Postgres or Neon
Storage: Cloudflare R2
Frontend Dashboards: Vercel

Production later:

Backend: AWS / DigitalOcean / Hetzner
Database: Managed PostgreSQL
Cache: Managed Redis
Storage: S3-compatible storage

29. Third-Party Integrations

Required for MVP

1. Maps provider
2. Push notifications
3. File/image storage
4. SMS or OTP provider, if phone login is used

Useful Later

1. WhatsApp Business API
 2. Online payment gateway
 3. Email service
 4. Analytics tools
 5. Error monitoring
 6. PDF generation service
 7. Route optimization service
-

30. Security Requirements

Authentication

- Passwords must be hashed.
- Use JWT access tokens.
- Use refresh tokens.
- Support token expiration.
- Add logout/token invalidation.

Authorization

- Role-based access control.
- Entity-level access control.
- Store owners can access only their own stores.
- Providers can access only their own providers.
- Drivers can access only assigned deliveries or available jobs.
- Clients can access only their own orders.

Data Protection

- Never expose password hashes.
- Hide sensitive driver/client details unless needed.
- Validate all API inputs.

- Use HTTPS.
- Rate-limit login and OTP endpoints.
- Log admin actions.

Payment Safety

- For cash on delivery, track who collected the cash.
 - Store cash collection amount.
 - Store payment status history.
 - Add dispute handling later.
-

31. Validation Rules

Order Validation

Before creating an order:

- Store must be active.
- Store must be open or accepting orders.
- Products must exist.
- Products must belong to selected store.
- Products must be available.
- Inventory must be enough.
- Delivery address must be valid.
- Total must be calculated by backend, not trusted from client app.

Inventory Validation

- Quantity cannot go below zero unless admin override is allowed.
- Every stock change should create an inventory movement.
- Product price changes should not affect old orders.

Delivery Validation

- Driver must be approved.
- Driver must be available.
- Delivery job must not already be assigned.
- Driver can update only assigned delivery.

Receipt Validation

- Receipt should be generated from backend values.
- Receipt number must be unique.
- Receipt should keep snapshots of totals.

32. Logging & Audit

The system should log important actions.

Actions to Log

- User login.
- Store approval/rejection.
- Driver approval/rejection.
- Order accepted/rejected.
- Inventory adjusted.
- Delivery assigned.
- Payment marked collected.
- Admin status changes.
- Refunds later.
- Manual changes by admin.

Suggested table:

```
audit_logs
- id
- actor_user_id
- action
- target_type
- target_id
- old_value
- new_value
- ip_address
- created_at
```

33. Error Handling

The API should return clear error messages.

Examples:

```
{
  "message": "Product is out of stock",
```

```
"code": "PRODUCT_OUT_OF_STOCK"
}
```

```
{
  "message": "You do not have permission to access this store",
  "code": "FORBIDDEN_STORE_ACCESS"
}
```

Common error codes:

```
UNAUTHORIZED
FORBIDDEN
VALIDATION_ERROR
NOT_FOUND
PRODUCT_OUT_OF_STOCK
STORE_CLOSED
ORDER_ALREADY_ACCEPTED
DELIVERY_ALREADY_ASSIGNED
PAYMENT_ALREADY_COLLECTED
```

34. MVP Features

The MVP should stay focused.

Must Have

Client

- Register/login.
- Browse stores.
- Browse products.
- Add to cart.
- Place order.
- Track order.
- View receipt.

Store

- Store login.
- Product management.
- Basic inventory.
- Accept/reject orders.

- Mark preparing/ready.
- Request delivery.
- View receipts.

Delivery

- Driver login.
- Driver availability.
- View available jobs.
- Accept job.
- Update delivery status.
- Mark cash collected.

Admin

- View users.
- Approve stores.
- Approve drivers.
- View orders.
- View deliveries.
- View receipts.

Backend

- Authentication.
- Authorization.
- Orders.
- Inventory.
- Deliveries.
- Receipts.
- Payments as cash.
- Notifications.

35. Should Wait Until Later

Do not build these in the first MVP unless absolutely needed:

- Multi-store checkout.
- Advanced route optimization.
- Full in-app chat.
- Wallet system.
- Online card payments.
- Loyalty system.
- Coupon system.
- AI recommendations.
- Barcode scanning.

- Advanced accounting.
 - Driver bidding.
 - Subscription plans.
 - Complex analytics.
-

36. MVP Development Order

Recommended development order:

Step 1: Backend Foundation

- Project setup.
- Database setup.
- Authentication.
- Roles and permissions.
- User model.

Step 2: Store Foundation

- Store model.
- Product model.
- Inventory model.
- Store dashboard APIs.

Step 3: Client Ordering

- Public store/product APIs.
- Cart logic on frontend.
- Create order API.
- Order status tracking.

Step 4: Store Order Management

- Store receives order.
- Accept/reject order.
- Inventory update.
- Mark preparing/ready.

Step 5: Delivery

- Driver model.
- Available jobs.
- Accept delivery.
- Delivery status updates.

- Cash collection.

Step 6: Receipts & Payments

- Generate receipts.
- Payment status.
- Cash collection records.

Step 7: Admin Dashboard

- User management.
- Store approval.
- Driver approval.
- View orders/deliveries/receipts.

Step 8: Service Provider Flow

- Provider profile.
- Services.
- Service requests.
- Quotes.
- Provider receipts.

37. Recommended Repository Structure

Option 1: Monorepo

```
platform/  
  apps/  
    client-app/  
    driver-app/  
    business-dashboard/  
    admin-dashboard/  
    backend/  
  
  packages/  
    shared-types/  
    ui/  
    config/
```

Option 2: Separate repositories

```
client-app  
driver-app  
business-dashboard  
admin-dashboard  
backend
```

Recommended:

```
Monorepo for early development
```

Why:

- Shared types.
- Easier coordination.
- Easier API contract sharing.
- Better for one developer or small team.

38. Environment Setup

Recommended environments:

```
local  
development  
staging  
production
```

Environment Variables

Backend:

```
DATABASE_URL  
REDIS_URL  
JWT_SECRET  
JWT_REFRESH_SECRET  
FCM_SERVER_KEY  
MAPS_API_KEY
```

```
STORAGE_ACCESS_KEY
STORAGE_SECRET_KEY
STORAGE_BUCKET
SMS_PROVIDER_KEY
```

Frontend:

```
API_BASE_URL
MAPS_PUBLIC_KEY
FCM_PUBLIC_CONFIG
```

39. Testing Plan

Backend Tests

- Auth tests.
- Role permission tests.
- Order creation tests.
- Inventory update tests.
- Delivery assignment tests.
- Payment collection tests.
- Receipt generation tests.

Frontend Tests

- Login flow.
- Order flow.
- Store order management flow.
- Driver delivery flow.
- Admin approval flow.

Manual MVP Test Scenario

1. Admin approves a store.
2. Store adds products and inventory.
3. Client places order.
4. Store accepts order.
5. Inventory decreases.
6. Store requests delivery.
7. Driver accepts delivery.
8. Driver picks up order.
9. Driver delivers order.

10. Driver collects cash.
11. Receipt becomes paid.
12. Admin sees completed order.

This is the most important MVP test.

40. Design Requirements

The next UI/UX design should be based on this file.

Design should include:

- Client app pages.
- Driver app pages.
- Store dashboard pages.
- Provider dashboard pages.
- Admin dashboard pages.
- User flows.
- Status transitions.
- Empty states.
- Loading states.
- Error states.
- Permission-based views.

Important Design Notes

- Client app should be simple and fast.
 - Driver app should be clear and usable while moving.
 - Store dashboard should focus on orders and inventory.
 - Provider dashboard should focus on requests and quotes.
 - Admin dashboard should focus on monitoring and control.
-

41. Important Business Rules

Store Order Rules

- Client cannot order unavailable product.
- Client cannot order more than available stock.
- Store can reject order with reason.
- Inventory should decrease after store accepts order, not only when client adds to cart.
- If order is cancelled before pickup, stock should be restored.

Delivery Rules

- Delivery should not be assigned before store accepts order.
- Driver can only have limited active deliveries at once.
- Driver cannot mark delivered before pickup.
- Cash collection must match order total if required.

Receipt Rules

- Receipt should be generated after order/service/delivery reaches payment-confirmed or completed status.
- Receipt should keep historical price values.
- Receipt should not change if product price changes later.

Authorization Rules

- Every API must check role.
 - Every business API must check entity ownership.
 - Store A cannot access Store B data.
 - Provider A cannot access Provider B data.
 - Driver cannot access unassigned delivery details after job is taken by someone else.
-

42. Future Features

Version 2

- Service provider flow fully expanded.
- In-app chat.
- Driver live tracking.
- Better reports.
- Coupons.
- Ratings.
- Notifications center.

Version 3

- Online payments.
- Wallet.
- Store subscription plans.
- Advanced inventory.
- Barcode scanning.
- Multi-store checkout.

Version 4

- Route optimization.
- AI recommendations.
- Demand forecasting.
- Advanced analytics.
- Accounting integrations.

43. Final Implementation Summary

The platform should be built around one shared backend and multiple role-based systems.

The first version should focus on this main flow:

```
Client places order
↓
Store accepts order
↓
Inventory updates
↓
Delivery is assigned
↓
Driver delivers order
↓
Cash is collected
↓
Receipt is generated
↓
Admin can monitor everything
```

The most important technical foundations are:

1. Strong authentication.
2. Role-based authorization.
3. Clean database design.
4. Controlled order and delivery statuses.
5. Reliable inventory updates.
6. Receipt generation.
7. Push notifications.
8. Admin monitoring.

Once this is working, the platform can expand safely into service providers, custom delivery, advanced reporting, online payments, and multi-store operations.

44. Feature Expansion Updates from Uploaded Notes

The uploaded feature update notes add a stronger product direction: the platform should not be treated only as a delivery app. It should become a **daily local problem-solving platform** for Lebanon.

The expanded direction is built around three value pillars:

1. **Certainty**
2. **Convenience**
3. **Coordination**

These pillars should influence the client app, store system, service provider system, delivery system, admin dashboard, APIs, and future roadmap.

45. Updated Product Positioning

Previous Positioning

The platform connects clients, stores, service providers, and delivery drivers through one shared backend.

Updated Positioning

The platform should become:

A local commerce, services, and delivery coordination system that helps people get products, solve urgent needs, request services, and trust the process from request to payment.

The app should help people:

- Find what they need faster.
 - Know if items are really available.
 - Trust the delivery process.
 - Reduce phone call confusion.
 - Track payments clearly.
 - Reach stores, providers, and drivers easily.
 - Request services and urgent deliveries from one place.
-

46. Three Main Product Value Pillars

46.1 Certainty

The app should reduce uncertainty for clients, stores, providers, and drivers.

Users should clearly know:

- What is available.
- When it will arrive.
- What it will cost.
- Who is delivering it.
- What stage the order is in.
- Whether there is a delay.
- Whether cash payment was collected.

Certainty features include:

- Smart ETA.
 - Stock certainty badges.
 - Visual order timeline.
 - Driver trust profile.
 - Clear receipt and payment status.
 - Delay warnings.
 - Delivery confidence indicators.
-

46.2 Convenience

The app should help users solve daily needs from one place.

Users should be able to:

- Order products.
- Request services.
- Send packages.
- Reorder common items.
- Schedule deliveries.
- Request urgent delivery.
- Ask for help finding something.

Convenience features include:

- One-tap reorder.
- Favorite stores.
- Saved shopping lists.

- Scheduled deliveries.
 - Urgent needs page.
 - Request Anything / Concierge Mode.
 - Help Me Find It search mode.
-

46.3 Coordination

The app should reduce confusion between clients, stores, providers, drivers, and admins.

Coordination features include:

- Status updates.
 - Push notifications.
 - Order timeline.
 - WhatsApp/call fallback.
 - Delivery issue flows.
 - Admin monitoring dashboard.
 - Cash collection tracking.
 - Support escalation.
-

47. Customer Experience Upgrades

47.1 Smart ETA and Delivery Confidence

The app should show more than basic statuses.

The client should see:

- Estimated preparation time.
- Estimated driver arrival time.
- Estimated delivery window.
- Delay warnings.
- Confidence level.

Example confidence labels:

High confidence

Possible delay

Busy period

API / Backend Impact

Add ETA fields to order and delivery responses:

```
estimated_preparation_minutes  
estimated_pickup_time  
estimated_delivery_time  
eta_confidence  
possible_delay_reason
```

Possible endpoint:

```
GET /api/v1/client/orders/:id/eta  
GET /api/v1/deliveries/:id/eta
```

Future module:

```
ETA & Prediction Module
```

47.2 Stock Certainty Badges

Products should clearly show availability.

Examples:

```
In stock  
Low stock  
Only 3 left  
Usually available  
Out of stock
```

This improves trust because clients can see whether the product is really available.

API / Backend Impact

Product listing APIs should return availability status:

```
stock_status  
available_quantity_display
```

```
is_low_stock
is_out_of_stock
usually_available
```

Possible values:

```
in_stock
low_stock
only_few_left
usually_available
out_of_stock
unavailable
```

47.3 Better Order Timeline UI

Order tracking should use a visual timeline.

Example timeline:

```
Order received
Store accepted
Preparing
Driver assigned
Picked up
On the way
Delivered
Completed
```

API / Backend Impact

Add timeline endpoint:

```
GET /api/v1/client/orders/:id/timeline
GET /api/v1/deliveries/:id/timeline
```

Timeline item structure:

```
{
  "status": "preparing",
  "title": "Store is preparing your order",
```

```
"description": "The store accepted your order and is preparing it.",  
"timestamp": "2026-05-01T12:00:00Z",  
"completed": true  
}
```

47.4 Driver Trust Profile

During delivery, the client should see basic driver information.

Possible fields:

- Driver first name.
- Profile photo.
- Rating.
- Vehicle type.
- Plate number if appropriate.
- Live location after pickup.

API / Backend Impact

Delivery tracking response should include:

```
driver_first_name  
driver_profile_photo  
driver_rating  
vehicle_type  
vehicle_plate_last_digits  
current_location_after_pickup
```

Important privacy note:

Do not expose unnecessary personal driver details.

47.5 Beautiful Receipt Experience

Receipt screens should be useful and clean, not only administrative.

Add:

- Summary card.
- Payment status badge.
- Delivery summary.

- Reorder button.
- Report issue button.
- Share receipt later.

API / Backend Impact

Receipt details should include:

```
receipt_summary  
payment_badge  
reorder_available  
related_delivery_summary  
issue_reporting_available
```

Possible endpoints:

```
POST /api/v1/client/orders/:id/reorder  
POST /api/v1/receipts/:id/report-issue
```

48. Lebanon Market Problem-Solving Features

48.1 Call and WhatsApp Fallback

Because many Lebanese users and businesses still depend on direct communication, the app should include fallback contact options.

Add buttons for:

- Call store.
- Call driver.
- Call provider.
- Open WhatsApp chat.
- Send prefilled WhatsApp message with order/request ID.

Example WhatsApp Message

```
Hello, I am contacting you about Order #ORD-000928 from the app.
```

API / Backend Impact

Contact info should be permission-based.

Possible endpoint:

```
GET /api/v1/orders/:id/contact-options
GET /api/v1/service-requests/:id/contact-options
GET /api/v1/deliveries/:id/contact-options
```

Response example:

```
{
  "can_call_store": true,
  "can_call_driver": true,
  "can_whatsapp_store": true,
  "whatsapp_prefilled_message": "Hello, I am contacting you about Order
#ORD-000928."
}
```

48.2 Cash on Delivery Enhancements

COD should be clear and safe.

Add:

- Exact amount to prepare.
- Need change toggle.
- Driver confirms collected amount.
- Store/admin cash collection visibility.
- Proof of delivery for expensive orders later.
- Dispute reporting for payment issues.

API / Backend Impact

Order checkout should include:

```
needs_change
cash_amount_client_has
amount_to_collect
```

Driver cash collection endpoint should support exact collected amount:

```
POST /api/v1/driver/deliveries/:id/cash-collected
```

Body example:

```
{
  "collected_amount": 12.50,
  "needs_change": false,
  "note": "Client paid exact amount"
}
```

Admin cash monitoring:

```
GET /api/v1/admin/cash-collections
GET /api/v1/admin/drivers/:id/cash-collections
```

48.3 Emergency and Urgent Categories

Add a dedicated urgent needs section.

Examples:

- Pharmacy now.
- Baby items.
- Water and gas.
- Urgent groceries.
- Forgotten item delivery.
- Document delivery.
- Key delivery.
- Repair pickup.
- Laundry pickup.

UI Impact

Client app should include:

```
Urgent Needs Page
Emergency Categories Section on Home
Quick Essentials Section
```

API Impact

```
GET /api/v1/client/urgent-categories
POST /api/v1/client/urgent-requests
```

48.4 Request Anything / Concierge Mode

Allow users to submit a request even if the product or service is not listed.

Flow:

```
Client writes what they need
↓
Client uploads photo if needed
↓
Client sets budget if needed
↓
Nearby stores/providers/admin receive request
↓
Relevant business responds
↓
Client confirms
↓
Order/service/delivery is created
```

Why It Matters

Many local businesses do not have full digital catalogs, especially at the beginning.

API Impact

```
POST /api/v1/client/concierge-requests
GET /api/v1/client/concierge-requests
GET /api/v1/client/concierge-requests/:id
POST /api/v1/store-owner/concierge-requests/:id/respond
POST /api/v1/provider-owner/concierge-requests/:id/respond
GET /api/v1/admin/concierge-requests
```

Future module:

48.5 Help Me Find It Search Mode

Add guided intent-based search.

Examples:

```
I need printer ink near me
I need a technician today
I need someone to pick up my phone for repair
I need medicine urgently
```

The system should route the user to:

- Matching stores.
- Matching service providers.
- Custom delivery option.
- Urgent support flow.

UI Impact

Add:

```
Help Me Find It Page
Guided Search Flow
Search Results with Suggested Actions
```

API Impact

```
POST /api/v1/client/help-me-find-it
```

Body example:

```
{
  "text": "I need printer ink near me",
  "location": {
    "lat": 33.8938,
    "lng": 35.5018
  }
}
```

```
}  
}
```

Response should include suggested paths:

```
{  
  "suggestions": [  
    {  
      "type": "store",  
      "title": "Stationery stores near you"  
    },  
    {  
      "type": "custom_delivery",  
      "title": "Ask a driver to pick it up"  
    }  
  ]  
}
```

49. Retention and Repeat Use Features

49.1 One-Tap Reorder

Users should reorder from:

- Previous orders.
- Previous stores.
- Receipt details.
- Favorites.

API:

```
POST /api/v1/client/orders/:id/reorder
```

Important rule:

When reordering, the backend must re-check:

- Product availability.
 - Current price.
 - Current stock.
 - Store open status.
-

49.2 Favorite Stores and Saved Lists

Users should be able to:

- Favorite stores.
- Save products.
- Create shopping lists.
- Save weekly essentials.
- Save repeat delivery requests.

API:

```
GET    /api/v1/client/favorites
POST   /api/v1/client/favorites
DELETE /api/v1/client/favorites/:id

GET    /api/v1/client/saved-lists
POST   /api/v1/client/saved-lists
PATCH /api/v1/client/saved-lists/:id
DELETE /api/v1/client/saved-lists/:id
POST   /api/v1/client/saved-lists/:id/items
DELETE /api/v1/client/saved-lists/:id/items/:itemId
```

Future module:

Favorites and Saved Lists Module

49.3 Scheduled Deliveries and Time Slots

Users should choose:

- Deliver now.
- Deliver later today.
- Schedule tomorrow.
- Choose a time slot.

API:

```
GET /api/v1/stores/:id/delivery-time-slots
GET /api/v1/service-providers/:id/time-slots
```

```
POST /api/v1/client/orders
POST /api/v1/client/service-requests
```

Add to order/service request:

```
fulfillment_type: now | scheduled
scheduled_for
scheduled_time_slot_id
```

Future module:

```
Scheduled Delivery Module
```

49.4 Family / Household Accounts

Allow shared household usage.

Examples:

- One home account.
- Multiple saved recipients.
- One person orders, another receives.
- Shared address notes.

Future API:

```
GET /api/v1/client/households
POST /api/v1/client/households
POST /api/v1/client/households/:id/members
DELETE /api/v1/client/households/:id/members/:memberId
```

This should be V2, not MVP.

49.5 Group Orders

Group orders can support:

- Office lunch.
- Family order.
- Shared household cart.

This is a later feature after the platform is stable.

50. Discovery and Premium UI Features

50.1 Smarter Home Screen Sections

Instead of only listing stores, the home screen should show helpful sections.

Examples:

```
Nearby stores
Fastest delivery
Open now
Best rated near you
Popular this week
Pharmacy now
Quick essentials
Send package
Repair and laundry
Urgent needs
Reorder again
```

API:

```
GET /api/v1/client/home
```

The home API should return dynamic sections:

```
{
  "sections": [
    {
      "type": "nearby_stores",
      "title": "Nearby stores",
      "items": []
    },
    {
      "type": "urgent_needs",
      "title": "Urgent needs",
      "items": []
    }
  ]
}
```

```
]
}
```

50.2 Better Store Pages

Store pages should quickly answer the customer's key questions.

Show:

- Delivery time.
- Average preparation time.
- Minimum order.
- Delivery fee.
- Open/closed badge.
- Top products.
- Popular items.
- Stock certainty.
- Reviews.
- Categories.

50.3 Better Provider Pages

Provider pages should show:

- Service type.
- Response time.
- Pricing style.
- Rating.
- Open/closed status.
- Available today badge.
- Past reviews.
- Pickup/delivery support.

50.4 Helpful Empty States

When there are no results, the app should suggest next actions.

Examples:

```
No nearby stores found → Try custom delivery
Product unavailable → Request anything
```

Store closed → Schedule for later
No provider available → Help me find it

This makes the app feel smart and helpful.

51. Store-Focused Feature Updates

51.1 Better Inventory Tools

Add:

- Low-stock alerts.
- Quick stock adjustment buttons.
- Duplicate product.
- Bulk import later.
- Hide unavailable items automatically.
- Best sellers insight.

API additions:

```
POST /api/v1/store-owner/stores/:storeId/products/:productId/duplicate
POST /api/v1/store-owner/stores/:storeId/products/bulk-import
GET /api/v1/store-owner/stores/:storeId/insights/best-sellers
GET /api/v1/store-owner/stores/:storeId/inventory/alerts
```

Bulk import should be V2.

51.2 Missed Sales Recovery Insights

Store owners should see:

- Cancelled order reasons.
- Rejected order reasons.
- Frequently searched but unavailable items.
- Low-stock lost opportunities.
- Products often bought together.

API:

```
GET /api/v1/store-owner/stores/:storeId/insights/missed-sales
GET /api/v1/store-owner/stores/:storeId/insights/unavailable-searches
GET /api/v1/store-owner/stores/:storeId/insights/bought-together
```

This should be V2.

51.3 Delivery Choice Recommendation

If a store supports both store drivers and platform drivers, the system can recommend:

- Use own driver.
- Use platform driver.
- Pickup only suggested.

Based on:

- Distance.
- Driver availability.
- Urgency.
- Store workload.
- Estimated delivery speed.

API:

```
GET /api/v1/store-owner/stores/:storeId/orders/:orderId/delivery-recommendation
```

Response example:

```
{
  "recommended_option": "platform_driver",
  "reason": "Platform drivers are available nearby and estimated delivery is faster."
}
```

This can start simple and become smarter later.

51.4 Better Staff Control

Add permission presets later:

```
Order manager
Inventory manager
Cashier
Delivery coordinator
Support role
```

API:

```
GET /api/v1/store-owner/stores/:storeId/permission-presets
POST /api/v1/store-owner/stores/:storeId/staff
PATCH /api/v1/store-owner/stores/:storeId/staff/:staffId/permissions
```

52. Driver-Focused Feature Updates

52.1 Better Job Information Before Acceptance

Before accepting a job, drivers should see:

- Estimated fee.
- Estimated distance.
- Pickup area.
- Dropoff area.
- Cash collection required or not.
- Item type if relevant.

API response for available jobs should include:

```
estimated_fee
estimated_distance
pickup_area
dropoff_area
cash_collection_required
item_type
estimated_duration
```

52.2 Better Delivery Exception Flows

Add structured flows for:

- Customer not answering.
- Wrong address.
- Store delayed.
- Item missing.
- Payment issue.
- Failed delivery reason.

API:

```
POST /api/v1/driver/deliveries/:id/report-issue
GET /api/v1/driver/deliveries/issue-reasons
```

Body example:

```
{
  "reason": "customer_not_answering",
  "note": "Called 3 times, no answer"
}
```

52.3 Driver Performance Score

Add later:

- On-time rate.
- Acceptance rate.
- Completion rate.
- Customer rating.
- Cash handling accuracy.
- Badge level.

API:

```
GET /api/v1/driver/performance
GET /api/v1/admin/drivers/:id/performance
```

This is V2/V3.

52.4 Driver Safety and Support

Possible additions:

- Emergency support button.
- One-tap support during active job.
- Issue escalation flow.
- Proof of delivery upload later.

API:

```
POST /api/v1/driver/deliveries/:id/emergency-support  
POST /api/v1/driver/deliveries/:id/proof-of-delivery
```

Proof of delivery should be later, especially for expensive orders.

53. Admin and Platform Intelligence Updates

53.1 Better Platform Monitoring

Admin dashboard should show cards for:

- Delayed orders.
- Failed deliveries.
- Cash not yet settled.
- Stores with frequent issues.
- Drivers with repeated failures.
- Low-stock alerts across stores later.

API:

```
GET /api/v1/admin/monitoring/delayed-orders  
GET /api/v1/admin/monitoring/failed-deliveries  
GET /api/v1/admin/monitoring/cash-unsettled  
GET /api/v1/admin/monitoring/problem-stores  
GET /api/v1/admin/monitoring/problem-drivers
```

53.2 Fraud and Risk Flags

Add internal warning flags later for:

- Repeated order cancellation.
- Repeated failed delivery.
- Suspicious cash mismatch.
- Bad rating patterns.
- Suspicious account behavior.

API:

```
GET /api/v1/admin/risk-flags
GET /api/v1/admin/risk-flags/:id
POST /api/v1/admin/risk-flags/:id/resolve
```

This should be advanced and not MVP.

53.3 Quality Control Scores

Track:

- Store response speed.
- Store completion rate.
- Provider response speed.
- Driver delivery speed.
- Support resolution time.

API:

```
GET /api/v1/admin/quality-scores
GET /api/v1/admin/stores/:id/quality-score
GET /api/v1/admin/providers/:id/quality-score
GET /api/v1/admin/drivers/:id/quality-score
```

This is V2/V3.

54. New Modules / Extensions Needed

The uploaded feature updates introduce these possible modules:

ETA & Prediction Module
Availability Badge Logic
WhatsApp / Contact Integration Layer
Scheduled Delivery Module
Favorites and Saved Lists Module
Urgent Request / Concierge Module
Help Me Find It Search Module
Trust and Ratings Enhancement Module
Driver Performance Module
Risk and Quality Monitoring Module

MVP should only include the simpler parts of these modules.

55. New Screens / Pages to Add

Client App

Urgent Needs Page
Favorites Page
Reorder Page
Scheduled Delivery Selector
Request Anything Page
Help Me Find It Page
Better Receipt Details Page
Improved Order Timeline Page
Driver Trust Profile Section

Store Dashboard

Missed Sales Insights Page
Stock Alerts View
Delivery Recommendation Panel
Best Sellers Insights
Improved Inventory Adjustment Modal

Driver App

Improved Job Details Screen
Delivery Issue Flow Screen
Support Escalation Screen

Performance Summary Page, later
Emergency Support Button

Admin Dashboard

Delay Monitoring Page
Cash Settlement Monitoring Page
Risk Flags Page, later
Quality Score Dashboard, later
Problem Stores and Drivers Page

56. Updated Feature Priority

56.1 Must Add Soon

These should be included in or near MVP because they improve trust and product value:

Smart ETA and confidence
Stock certainty badges
Better order timeline UI
Call and WhatsApp fallback
COD improvements with need-change option
One-tap reorder
Scheduled delivery / time slots
Emergency categories
Driver trust profile
Better store pages
Better provider pages
Helpful empty states

56.2 Strong V2 Features

Add after the core MVP is stable:

Request Anything / Concierge Mode
Help Me Find It intent-based search
Favorite stores and saved lists
Household accounts
Better driver exception flows

Missed sales insights for stores
Driver performance score
Better admin quality dashboards

56.3 Later / Advanced Features

Only after the platform proves product-market fit:

Group orders
Full in-app chat
AI recommendations
Loyalty system
Online card payments
Wallet system
Route optimization
Advanced analytics
Proof of delivery media tools
Fraud scoring system

57. Updated MVP Recommendation

The MVP should still stay focused, but it should include a few trust-building features from the uploaded notes.

MVP Core Flow

```
Client places order
↓
Store accepts order
↓
Inventory updates
↓
Delivery is assigned
↓
Driver delivers order
↓
Cash is collected
↓
Receipt is generated
```

↓

Admin can monitor everything

Add These Trust Features to MVP

Clear stock status
Visual order timeline
Basic ETA
Driver first name and vehicle type
Call buttons
WhatsApp fallback
Exact cash amount to prepare
Need change option
Receipt summary card
Emergency categories on home
Helpful empty states

These features make the first version feel much stronger without making the backend too complex.

58. Updated Implementation Order

Step 1: Backend Foundation

- Authentication.
- Authorization.
- Roles.
- Users.
- Entity ownership rules.

Step 2: Store, Product, and Inventory

- Store model.
- Product model.
- Inventory model.
- Stock status badges.
- Low-stock logic.

Step 3: Client Ordering

- Store listing.
- Better store page data.
- Cart and checkout.
- Need-change option for COD.

- Create order.

Step 4: Order Timeline and Notifications

- Order statuses.
- Timeline endpoint.
- Push notifications.
- Basic ETA fields.

Step 5: Store Order Management

- Accept/reject.
- Preparing/ready.
- Inventory update.
- Request delivery.
- WhatsApp/call contact options.

Step 6: Delivery

- Driver availability.
- Better job details.
- Accept job.
- Delivery status updates.
- Driver trust profile.
- Cash collection confirmation.

Step 7: Receipts

- Receipt generation.
- Payment status badge.
- Receipt details.
- Reorder endpoint.

Step 8: Admin Monitoring

- Approvals.
- Active orders.
- Active deliveries.
- Delayed orders.
- Failed deliveries.
- Cash collection visibility.

Step 9: Service Provider Flow

- Provider profiles.
- Services.
- Service requests.

- Quotes.
- Provider delivery support.

Step 10: V2 Problem-Solving Features

- Request Anything.
 - Help Me Find It.
 - Favorites.
 - Scheduled deliveries.
 - Missed sales insights.
 - Better driver issue flows.
-

59. Final Updated Product Principle

The app should not only ask:

How do we deliver things?

It should ask:

How do we reduce uncertainty, save time, improve trust, and solve local daily problems?

This direction makes the platform:

- More useful.
- More attractive.
- More trusted.
- More sticky.
- More needed in the Lebanese market.